Identifying LLMs from Their Responses

CMPSC 448 Midterm Project

Laasya Sree Ayapati

1. Problem Definition and Dataset

The aim is to determine the LLM family that produced the response. With a prompt and a response, I train a classifier to predict the LLM family. I pose two key questions. RQ1: Is the response text alone sufficient for classifying the LLM? RQ2: Does the prompt assist more than the response text or the prompt alone? I have also conducted RQ3 (can it be generalized to unseen task types) and RQ4 (what are the discriminating features of the models).

Dataset source. I use the publicly available LMSYS Chatbot Arena Conversations dataset from Hugging Face (lmsys/chatbot_arena_conversations, 33K conversations). In this dataset, a prompt written by a real user is answered by two anonymous LLMs, and thus each conversation provides two labeled (LLM, input, output) examples. I filter out non-English conversations, take the first turn (the first user prompt and its response) in each conversation, and discard responses less than five words long. I have collected no private data.

The families for LLMs. I sorted the raw model name into four families: gpt (gpt-4, gpt-3.5-turbo), claude

(claude-v1, claude-instant-v1 and related), palm (palm-2), and llama (Llama and Llama-derived chat models like Vicuna, Koala, Alpaca). There were 33,642 samples after filtering, but the samples were imbalanced (llama 17,748; gpt 7,671; claude 5,600; palm 2,623). In order to make the samples balanced, I sampled 1,000 examples per family, making up 4,000 in total.

Splits. Stratifed 80/10/10 split with a fixed seed: 3,200 train, 400 validation, 400 test (100 per family in the test set, which means chance accuracy is 25%). For RQ3, I label each prompt as coding, math, writing, or qa using a keyword-based heuristic. Among the 4,000 samples, there are 2,887 qa, 458 writing, 409 coding, 246 math.

2. Models and Training

Preprocessing. Lowercased, tokenized using a regular expression into words and punctuation tokens, vocabulary of the 20,000 most frequent training tokens (including PAD, UNK, SEP). Responses are truncated to 256 tokens and prompts to 64.

For the input+output version, I concatenate prompt, SEP, and response into one sequence.

CNN. Embedding (dimension 128), three parallel 1D convolutions with filter sizes 3, 4, 5, 100 filters for each,

ReLU, max-pooling over time, concatenation (300 features), dropout 0.5, and then a linear layer to 4 classes.

RNN. Embedding (dimension 128), one bidirectional LSTM layer (hidden dimension 128 per direction), masked mean pooling over the non-padding positions, dropout 0.5, and then a linear layer to 4 classes.

Training. Adam optimizer, learning rate 1e-3, weight decay 1e-5, batch size 64, 12 epochs, cross-entropy loss. I save the weights of the epoch with the highest validation accuracy and use the test accuracy and macro F1 of this checkpoint. All models share the same seed (0). All models were trained on a CPU (MacBook Air).

3. Results (RQ1 and RQ2)

Test accuracy and macro F1 (4 classes, chance = 25%, n = 400 test examples).

Input to classifier	CNN acc	CNN F1	BiLSTM acc	BiLSTM F1
Output only (RQ1)	63.3%	0.627	54.3%	0.526
Input (prompt) only	27.8%	0.269	27.0%	0.259
Input + output	61.5%	0.616	56.8%	0.543

RQ1. On its own, the response is sufficient for the LLM family recognition much higher than chance. CNN got 63.3%, and BiLSTM 54.3%. The CNN outperformed BiLSTM by 9%. With 400 test samples, one accuracy measure has approximately +/- 2.5 of sampling noise, so I consider only differences of 5+ points significant.

RQ2. Prompt on its own performs near chance (27.8% and 27.0%). Combining the prompt with the response did not bring noticeable improvement: the CNN result decreased by 1.8%, and BiLSTM increased by 2.5%, both values within the noise. It is expected for this dataset. In Chatbot Arena, the same prompt is provided to two models, so the prompt has almost zero informational value to recognize the correct answerer. The slight non-chance signal in the input-only can be because of some prompt themes or styles differences between model groups. The prompts do not provide useful

information for this task, but the response does.

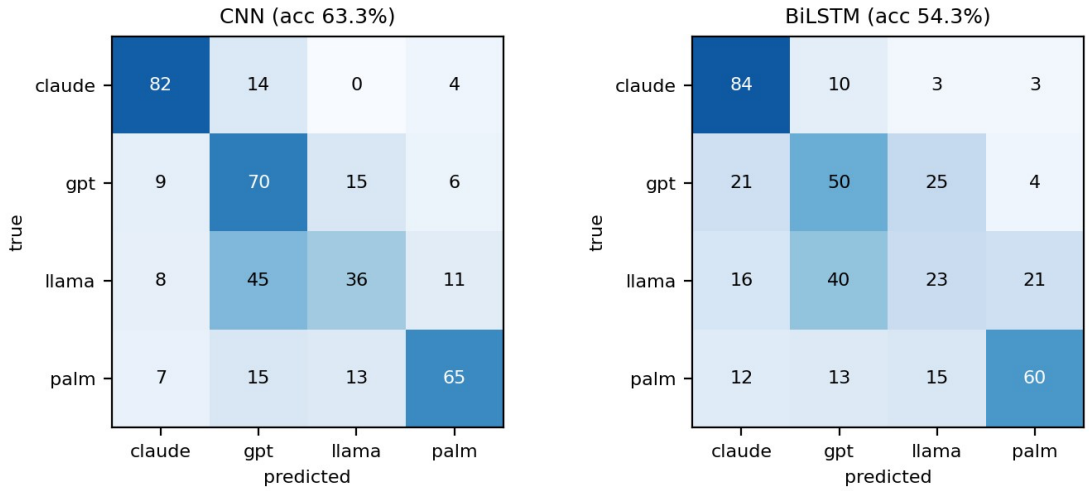


Figure 1. Confusion matrices on the test set, response only (rows = true class, 100 per class).

Per-class recall for the CNN: claude 82%, gpt 70%, palm 65%, llama 36%. The class that performs worst is llama, of which 45 out of 100 examples were classified as gpt. One possible explanation (not tested) is that the llama class consists of Llama-based chat models fine-tuned using ChatGPT data, hence they write like gpt. Claude is the easiest to recognize.

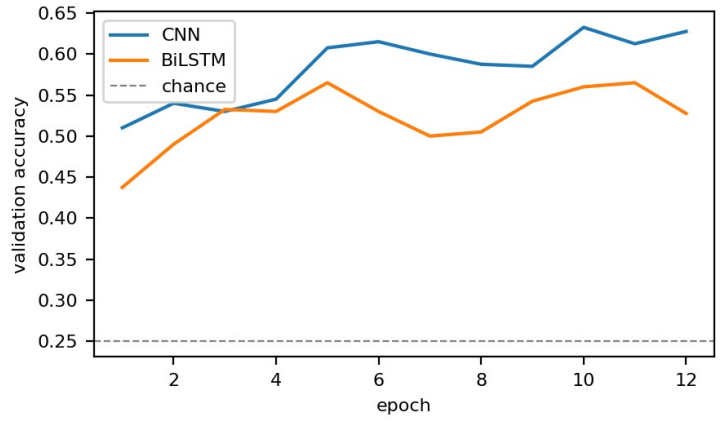


Figure 2. Validation accuracy per epoch, response only. The BiLSTM peaks early and fluctuates, while its training loss keeps dropping, which points to overfitting.

4. In-Depth Analyses

4.1 RQ3: generalization across tasks

The model has been trained on three datasets and tested on the remaining one (response only). A portion of the training dataset was set aside for validation. The in-distribution baseline from section 3 is 63.3% (CNN) and 54.3% (BiLSTM).

Held-out task	Test n	CNN acc	BiLSTM acc
coding	409	57.5%	45.2%
math	246	55.3%	48.0%
qa	2887	50.3%	43.7%
writing	458	57.9%	56.1%

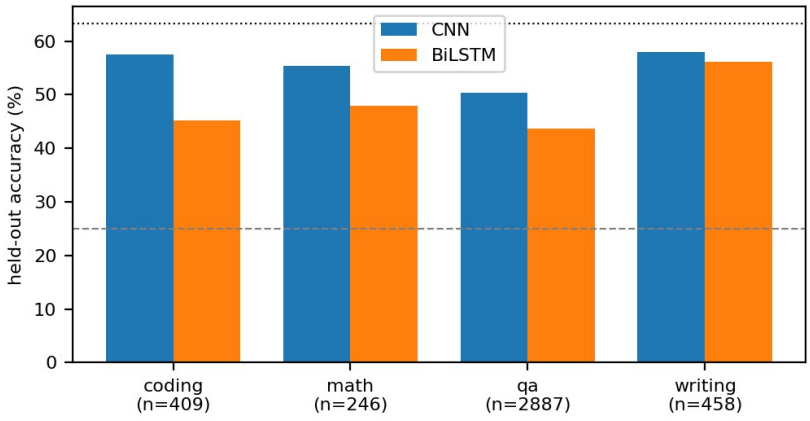


Figure 3. Held-out task accuracy. Dotted line = in-distribution CNN baseline, dashed = chance.

Accuracy on new tasks degrades, but remains way above 25% and hence, part of the fingerprint seems to generalize between tasks and part seems task-specific. In particular, the CNN accuracy on the coding task goes down to 57.5%, on the math task to 55.3%, on the writing task to 57.9% and finally to 50.3% on qa. However, the writing task generalizes the best as the BiLSTM only slightly changes its performance to 56.1% on the writing task. The QA task appears to be the worst case but it suffers from a confound in the fact that excluding the qa task removes 2,887 of the 4,000 examples, and the model is trained on just about 1,100 examples. The math and coding task have very little examples (246 and 409) and hence, the reported results seem to be noisy.

4.2 RQ4: what distinguishes the models

Average response statistics per family (on the balanced 4,000 examples):

Feature	claude	gpt	llama	palm
Characters	1023	852	735	976
Words	168.0	135.8	121.8	169.0
Type-token ratio	0.659	0.705	0.661	0.595
Newlines	14.8	10.1	6.6	12.0
Bold marks (**)	0.006	0.000	0.017	1.394
Bullet lines	1.03	0.42	0.34	1.94
Numbered lines	1.45	1.82	1.26	0.65
Exclamation marks	0.52	0.25	0.18	0.22
Avg word length	5.00	5.02	5.01	4.64

Clearly, the families exhibit distinct patterns. Claude provides the longest responses, which are marked by their most exclamation marks. Llama provides the shortest responses and fewest line breaks. Palm stands out in its formatting: it employs markdown for its bolds more than the other families (on average 1.4 per response vs near 0) and fewer words in its average word. The highest type/token ratios and most numbered lists appear in GPT.

In order to find out the contribution of structure towards the signal, a logistic regression was performed using just the above mentioned 13 manually crafted features, as well as ablation experiments on the neural families:

Setting (response only)	CNN acc	BiLSTM acc
Baseline (full text)	63.3%	54.3%
Formatting stripped (markdown, code blocks removed)	62.8%	59.5%
Truncated to first 60 words	59.5%	55.3%
13 handcrafted features + logistic regression	44.8%	n/a

Formatting and length account for 44.8%, which is already more than chance, so the signal must contain structural aspects (logistic regression weights are the highest for word count, bold marks, and character

count). However, the neural networks achieve 63% performance, thus utilizing words as well. The neural networks did not suffer from removing the formatting (BiLSTM model actually improved its performance by around 5%, probably due to cleaner input or random fluctuation), meaning that the neural networks do not rely on markdown format. Reduction of responses to 60 words decreased the performance of CNN by around 4 points, indicating that the length and later parts of a response add some value to the signal, though it is not what the neural networks use. In general, the linguistic signal appears to have a greater influence on the result than the structural one. No multiseed test was performed.

5. Lessons and Experience

Having more data was crucial. My quick test with 300 examples per family yielded about 51% for CNN with all over-the-top results. With 1,000 per family CNN scored 63%. The small sample also spoiled my ablative testing, making me pay attention to how large the testing set was before trying to make any conclusions from the comparison of several percentage points.

CNN performed better than LSTM on this data. Training loss for BiLSTM dropped while its validation accuracy was flat or random, meaning the model was overfitting. CNN pools the most influential local phrases, which suits a task when local stylistic patterns are crucial. With more data or pre-trained embeddings the performance gap between the two might diminish.

At first I assumed the prompt would leak the label. Initially my concern was input only being suspiciously good, until I analyzed the way dataset is built (since there are two instances per prompt, one for each model). I've also figured out that the same prompt could appear in the train and test dataset with different labels since the split is done on examples not prompts. It would have been cleaner if split was done on prompts.

Engineering issues. Pandas recently broke my balancing and statistics with groupby/apply changes (tuple key broke saving to JSON). Rewrote those parts with plain loops. Also had to log into Hugging Face and use their interface to download the data.

Limitations. One seed only, one split, so there are no confidence intervals. Models date from 2023-era. The llama family consists of several different models, while gpt-4 and gpt-3.5 are grouped together, so family boundaries are ambiguous. Task labels are purely heuristics based. First turn only was considered.

Future work. Repeat for multiple seeds and prompts, include newer models and more families, use pretrained embeddings or transformers baseline model, and test robustness to paraphrasing and normalizing conversation lengths.